Implementation and Benchmarking of a Simplified SPHINCS+-Style Post-Quantum Signature Scheme

1. Introduction

Post-quantum cryptography has attracted a great deal of attention because large-scale quantum computers would put serious pressure on many public-key systems that are widely used today. Signature schemes based on RSA or elliptic-curve assumptions are especially exposed to this concern. For that reason, hash-based signatures have become an important area of study, since their security relies mainly on the strength of the underlying hash function rather than on number-theoretic problems.

This project develops a proof-of-concept implementation of a simplified SPHINCS+-style signature system. The goal was not to reproduce the full standard in every detail. Instead, the project focused on the main ideas behind the design and on building a working modular prototype that could be tested and benchmarked in a clear way. The system combines three core components: WOTS+ as the one-time signature layer, a Merkle tree as the authentication layer, and FORS as the few-time message-dependent signing layer.

The implementation was written in Python and organised by modules so that each part could be developed, tested, and analysed separately before integration. In addition to the implementation itself, the project also included correctness testing and a range of benchmark experiments. Several optimisation ideas were explored as well, especially hash-chain caching in WOTS+ and authentication-path caching in FORS. Together, these results provide a practical demonstration of how a simplified SPHINCS+-style system can be assembled and evaluated.

2. Background

2.1 Hash-Based Signatures

Hash-based signatures form one of the main families of post-quantum digital signatures. Their security is tied to properties such as preimage resistance and collision resistance of cryptographic hash functions. This makes them especially appealing in the post-quantum setting, because no quantum attack is currently known that breaks secure hash functions in the same way Shor's algorithm threatens RSA and ECC.

At the same time, basic hash-based signatures are often restrictive. Many of them are one-time or few-time by nature, meaning that a key can only be used safely a limited number of times. To make such schemes practical, larger structures are usually needed so that many short-lived signing keys can be compressed and authenticated under a compact public value.

2.2 WOTS+

WOTS+ is a Winternitz one-time signature scheme based on hash chains. A message digest is converted into base-w digits, a checksum is appended, and those values decide how far to move along each chain. The public key consists of the chain endpoints. In practice, the parameter w controls a trade-off: smaller values tend to produce longer signatures, while larger values reduce the number of chains but increase the chain length.

2.3 Merkle Trees

A Merkle tree compresses a set of leaves into one root value by repeatedly hashing pairs of nodes. The root acts as a compact commitment to the full set. To authenticate one leaf, it is enough to provide that leaf together with its authentication path. In hash-based signature systems, this is a natural way to authenticate many one-time public keys under a single public root.

2.4 FORS

FORS, short for Forest of Random Subsets, is the few-time signature layer used in SPHINCS+-style designs. Rather than relying on one tree, it uses several small trees. A message determines one selected index in each tree, and the signer reveals the corresponding secret values and authentication paths. The resulting roots are then compressed into a compact public-key value.

2.5 SPHINCS+-Style Hierarchy

The main design idea behind SPHINCS+ is hierarchical composition. A lower message-dependent layer first handles information derived from the message, while an upper one-time signature and Merkle authentication layer ties that information to a compact public root. Our project follows this idea in simplified form. Although the implementation is smaller and more educational than the standard itself, the core structure is preserved.

3. System Design

The system is organised as a modular signing pipeline. Shared hash utilities and common settings provide a consistent foundation. On top of that, the WOTS+ module handles one-time signing, the Merkle tree module authenticates compressed WOTS+ public keys, and the FORS module signs message-dependent information. These parts are then connected by a top-level workflow that exposes key generation, signing, and verification.

The overall data flow is straightforward. A message is first processed by FORS. The output from FORS is then used as the input to the selected WOTS+ instance. The corresponding WOTS+ public key is compressed into a single Merkle leaf, and the Merkle authentication path proves that the chosen leaf belongs to the stored public root. During verification, the same path is followed in reverse: FORS is checked first, then the WOTS+ public key is reconstructed, compressed into a leaf, and authenticated back to the Merkle root.

The system was designed with four practical goals in mind. First, each module should be simple enough to understand and test on its own. Second, all modules should follow the same data and

hash conventions. Third, the integrated pipeline should remain logically consistent from end to end. Finally, the implementation should be suitable for benchmarking, so that performance trade-offs can be observed rather than only described in theory.

4. Implementation

4.1 Hash Utilities and Common Settings

All modules in the system share a unified hash function based on SHA-256. A helper function H(data) takes a bytes object as input and returns a 32-byte digest. Using one common hash function across the project keeps the implementation consistent and also makes later replacement easier if a different hash primitive were to be explored.

The output length n = 32 is used as the common security parameter throughout the project. Secret-key values, public-key elements, leaves, roots, and intermediate outputs are all represented as byte strings of that length. A shared data convention is also used for Merkle integration: when a WOTS+ public key must be inserted into the tree, it is first compressed as leaf = H(b".join(pk)). This ensures that all tree nodes have the same format and size.

4.2 WOTS+ Module

The WOTS+ module implements the Winternitz one-time signature scheme and acts as one of the core pieces of the full SPHINCS+-style system. It supports key generation, signing, and verification-related recovery of candidate public-key values. The module is configurable through the Winternitz parameter w, which allows different performance and size trade-offs to be explored.

During key generation, the module produces a private key as a list of random 32-byte values. Each value forms the start of a hash chain. The public key is derived by hashing each chain element w - 1 times. To improve efficiency, the implementation precomputes all intermediate values on each chain and stores them in a cache. This avoids repeating the same hash work every time a signature is generated.

To sign, the processed input is converted into base-w digits, a checksum is appended, and each resulting digit determines how far to advance on the corresponding hash chain. The chosen intermediate values form the signature. During verification, the remaining steps on each chain are applied in order to recover a candidate public key. In the integrated system, this reconstructed public key is then compressed into a Merkle leaf and authenticated through the Merkle tree.

Three implementation ideas were examined for WOTS+. First, a cached version was compared with a baseline version without caching. Second, several values of w were tested to study their impact on signature length and running time. Third, multiprocessing-based parallel execution was explored. The results showed that caching was clearly useful, that the parameter w exposed a meaningful trade-off, and that parallel execution was not worthwhile in this Python setting because process overhead outweighed the available gain.

4.3 Merkle Tree Module

The Merkle tree module compresses a set of leaves into a single root value that serves as a compact commitment to the entire set. In the SPHINCS+-style pipeline, this root authenticates the family of WOTS+ public keys without requiring all of them to be revealed individually.

The module is implemented as a Python class called MerkleTree. The tree is stored level by level, with leaves at the first level and the root at the final level. Four main methods are provided: build(leaves) constructs the tree, root() returns the root value, auth_path(idx) produces the authentication path for one selected leaf, and compute_root(leaf, auth_path, idx) reconstructs the root during verification.

For simplicity and determinism, whenever a level contains an odd number of nodes, the final node is duplicated so that every node can be paired. The module also performs basic input validation, including checks for empty leaf lists, invalid indices, and attempts to read the root before construction is complete. In the full system, the Merkle root becomes the compact public commitment for all WOTS+ one-time keys.

4.4 FORS Module

The FORS module implements the few-time signature layer of the system. It is the first message-dependent signing stage in the SPHINCS+-style design. Unlike WOTS+, which is strictly one-time, FORS signs by revealing one selected secret value from each of several small Merkle trees, making it a good fit for the lower layer of the hierarchy.

The module is parameterised by k, the number of trees, and a, the height of each tree. Each tree therefore has t = 2^a leaves. During key generation, t random 32-byte secret values are created for each tree, each value is hashed into a leaf, and a Merkle tree is built over those leaves. The roots from all k trees are concatenated and hashed together to form one compact public key.

To sign a message, the module hashes the message and derives k leaf indices from the resulting bit string. For each tree, the signer reveals the secret value at the chosen index together with the corresponding authentication path. Verification recomputes the same indices, rebuilds each leaf by hashing the revealed secret value, reconstructs each tree root, and hashes the roots together again. The signature is valid only if that reconstructed value matches the stored public key.

An optimised variant, FORSCached, was also implemented. In the baseline version, authentication paths are computed on demand during signing. In the cached version, all paths are precomputed during key generation and stored in memory. This makes key generation heavier and increases memory use, but it can significantly reduce signing time, especially for larger parameter settings.

4.5 Overall Signing and Verification Workflow

The top-level module integrates WOTS+, Merkle Tree, and FORS into one complete signing system. It exposes three main functions: keygen(), sign(), and verify(). These functions define the lifecycle of key generation, message signing, and verification for the full prototype.

During key generation, the system first creates multiple WOTS+ key pairs. Each WOTS+ public key is compressed into a Merkle leaf, and a Merkle tree is built over all such leaves. The Merkle

root then becomes the public commitment for the WOTS+ layer. Separately, one FORS key pair is generated. The private key stores the WOTS+ secret keys, WOTS+ caches, the Merkle tree, and the FORS private-key material. The public key stores the Merkle root and the FORS public key.

During signing, the message is first handled by FORS, which produces a FORS signature and a message-dependent intermediate value. That value is then signed by the chosen WOTS+ key pair at index idx. The system also retrieves the Merkle authentication path for the corresponding leaf. The final signature therefore contains the FORS signature, the WOTS+ signature, the Merkle authentication path, and the chosen index.

During verification, the FORS layer is checked first. If that step fails, the entire signature is rejected. Otherwise, the verifier reconstructs the intermediate value needed by WOTS+, uses the WOTS+ signature to recover a candidate public key, compresses it into a Merkle leaf, and recomputes the Merkle root using the supplied authentication path. The signature is accepted only if the reconstructed root matches the public root.

5. Testing and Experiments

5.1 Correctness Testing

Correctness testing was carried out at both the module level and the full-system level. The aim was to confirm that valid inputs were accepted, tampered inputs were rejected, and the modules continued to behave consistently once integrated.

5.1.1 WOTS+ Correctness Testing

The WOTS+ module was tested using valid signing and verification cases together with several tampering scenarios. These included message tampering, signature tampering, public-key tampering, and invalid-input handling. Valid message-signature pairs were accepted, while altered or malformed cases were rejected. These tests supported the correctness of the WOTS+

key generation and signing logic, as well as the candidate public-key reconstruction step used during verification.

5.1.2 Merkle Tree Correctness Testing

The Merkle tree module was tested with both normal and abnormal inputs. A basic four-leaf case confirmed that the original root could be reconstructed correctly from each leaf and its authentication path. An odd-leaf case confirmed that duplicating the final node behaved as intended. Additional tests covered empty trees, empty input lists, negative indices, and out-of-range indices. All valid cases passed successfully.

5.1.3 FORS Correctness Testing

The FORS module was tested under six categories of conditions: valid signature, wrong message, tampered secret value, tampered authentication path, tampered index, and wrong public key. The same categories were repeated for the cached FORS variant. A cross-check also confirmed that the cached and baseline implementations produced the same logical outputs for the same key and message. Across all of these tests, the module behaved as expected.

```
(base) yixuanwang@Mac tests % python test_fors.py
=== FORS Test Start ===
keygen done, public key: 7763acf45c79be8b...
sign done, indices: [55, 28, 53, 31, 24, 40, 45, 10, 24, 55]
verify result: True
verify with wrong message: False
=== FORS All Tests Passed ===
```

Figure 1. Terminal output of test_fors.py showing FORS correctness test results.

5.1.4 Full Pipeline Correctness Testing

The complete SPHINCS+-style pipeline was also tested end to end. A valid message-signature pair was accepted, while a different message checked against the same signature was rejected. These results confirmed that the interfaces between FORS, WOTS+, and the Merkle tree were

consistent and that root reconstruction during verification matched the original public key only when the signature was valid.

```
● (base) yixuanwang@Mac tests % python test_sphincs.py
  === Test Start ===
  keygen done, merkle root: cf596624afd7aa2d...
  sign done, idx=0
  verify result: True
  verify with wrong message: False
  === All Tests Passed ===
```

Figure 2. Terminal output of test_sphincs.py showing full pipeline correctness test results.

5.2 WOTS+ Benchmark and Analysis

A benchmark study was carried out for the WOTS+ module to evaluate both optimisation ideas and parameter trade-offs. Three aspects were examined: the difference between a baseline implementation and the cached implementation, the effect of different Winternitz parameters w, and the practical effect of multiprocessing-based parallel execution.

In the first experiment, the cached implementation was compared with a baseline version that did not store intermediate chain values. The cached version improved signing speed because repeated hash-chain computation was replaced by direct lookup. In the second experiment, several values of w were tested. Larger values generally improved raw signing speed and reduced the number of chains, but they also implied longer chains and higher memory cost when caching was used. In practice, w = 16 gave the best overall balance for this project.

In the third experiment, serial and multiprocessing-based parallel versions were compared. Although the chains are independent in theory, the parallel version did not outperform the serial one in practice. Process creation, serialization, and inter-process communication introduced too much overhead relative to the amount of useful work in each task. For that reason, parallelisation was not adopted in the final WOTS+ design.

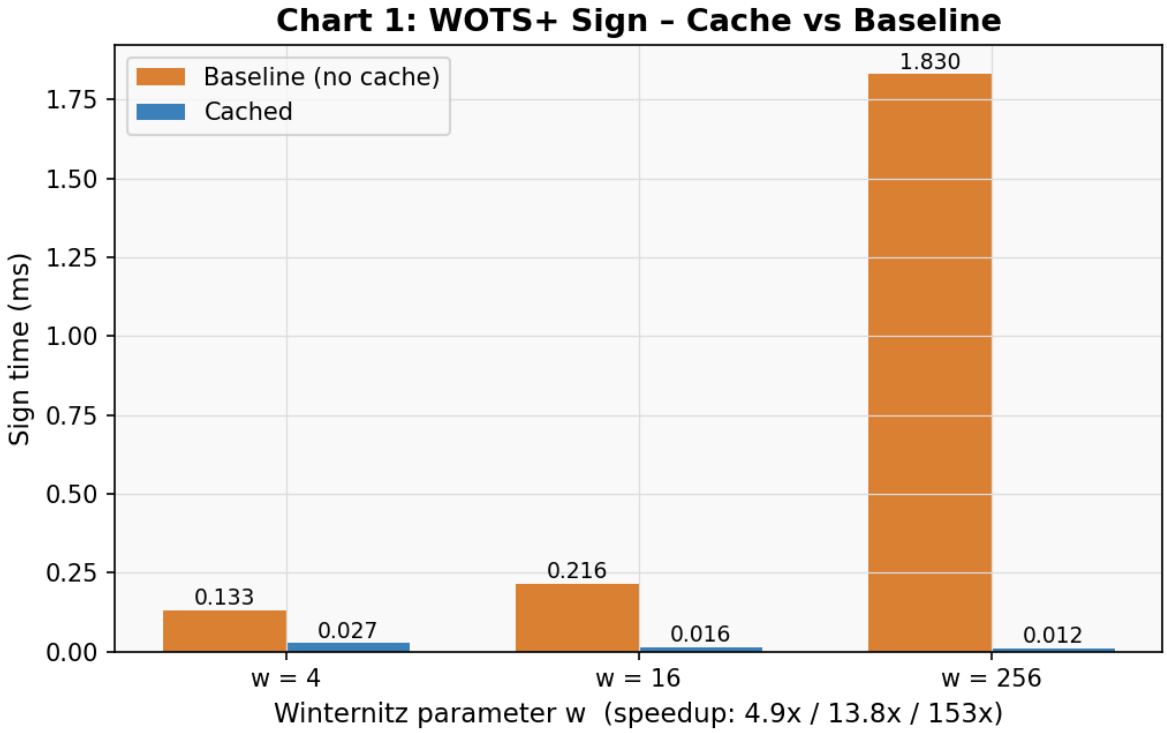

Figure 3. WOTS+ signing time with and without hash-chain caching across different Winternitz parameter settings.

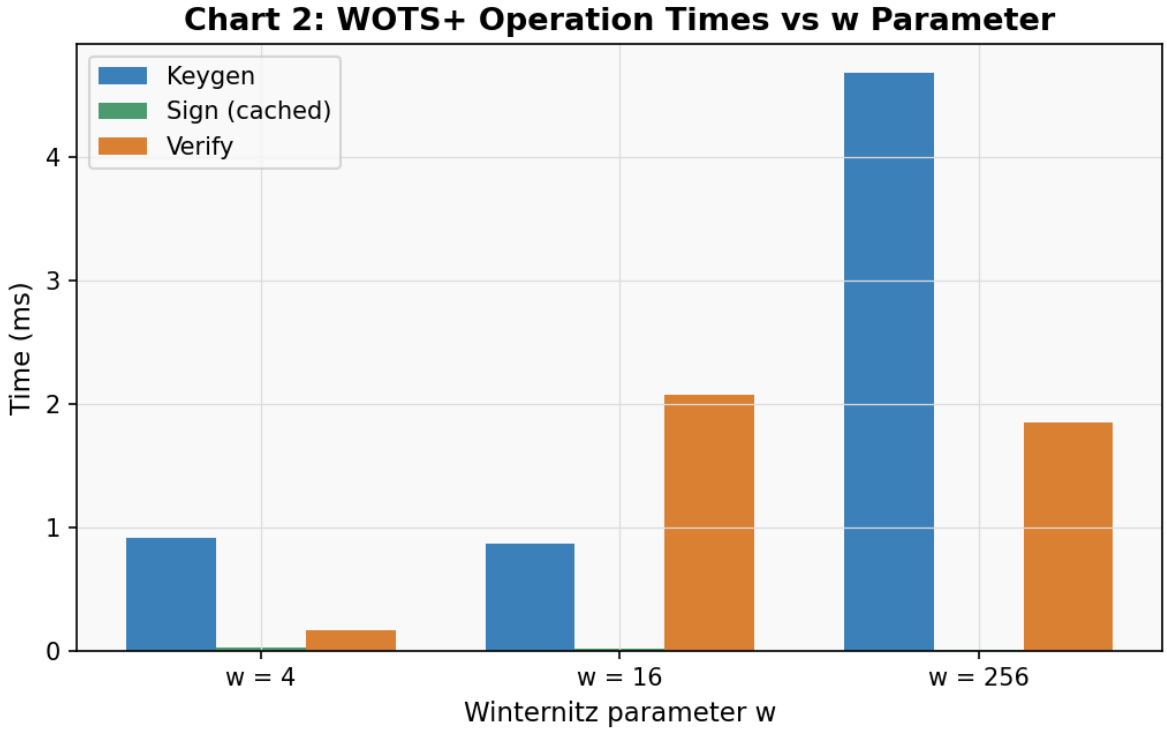

Figure 4. WOTS+ key generation, cached signing, and verification time across different Winternitz parameter settings.

5.3 Merkle Tree Benchmark

The Merkle tree benchmark measured the average running time of three operations: tree construction, authentication-path generation, and root reconstruction. The experiments were carried out with trees containing 4, 8, 16, and 32 leaves.

The trend was clear. As the number of leaves increased, all three operations became slower. Tree construction remained the most expensive step because it processes the whole tree, while authentication-path generation and root reconstruction were faster because they only involve one branch. Some small fluctuations appeared across repeated runs, but the general pattern stayed stable.

```
Merkle Tree Benchmark Results
======================================================================
Leaves    Build(s)          AuthPath(s)       ComputeRoot(s)
4         0.0000019196      0.0000004197      0.0000009952
8         0.0000036843      0.0000005192      0.0000013589
16        0.0000066516      0.0000005996      0.0000016052
32        0.0000118941      0.0000006584      0.0000016790
```

Figure 5. Benchmark results of the Merkle Tree module (to be inserted if a Merkle benchmark chart is available).

5.4 FORS Benchmark and Analysis

A separate benchmark compared the baseline FORS implementation with the cached variant FORSCached. The three measured operations were key generation, signing, and verification, evaluated across several parameter sets.

The results showed a predictable trade-off. The cached version increased key-generation time because all authentication paths had to be prepared in advance and stored in memory. However, it reduced signing time substantially because signing no longer needed to traverse the trees on demand. The benefit became larger as the parameters grew. Verification time remained broadly

similar between the two versions because verification still reconstructs roots from the given paths and does not make direct use of the cache.

Overall, authentication-path caching was an effective optimisation for the signing phase. The extra key-generation cost can be viewed as a one-time setup expense that becomes more acceptable when the same key pair is used for multiple signing operations.

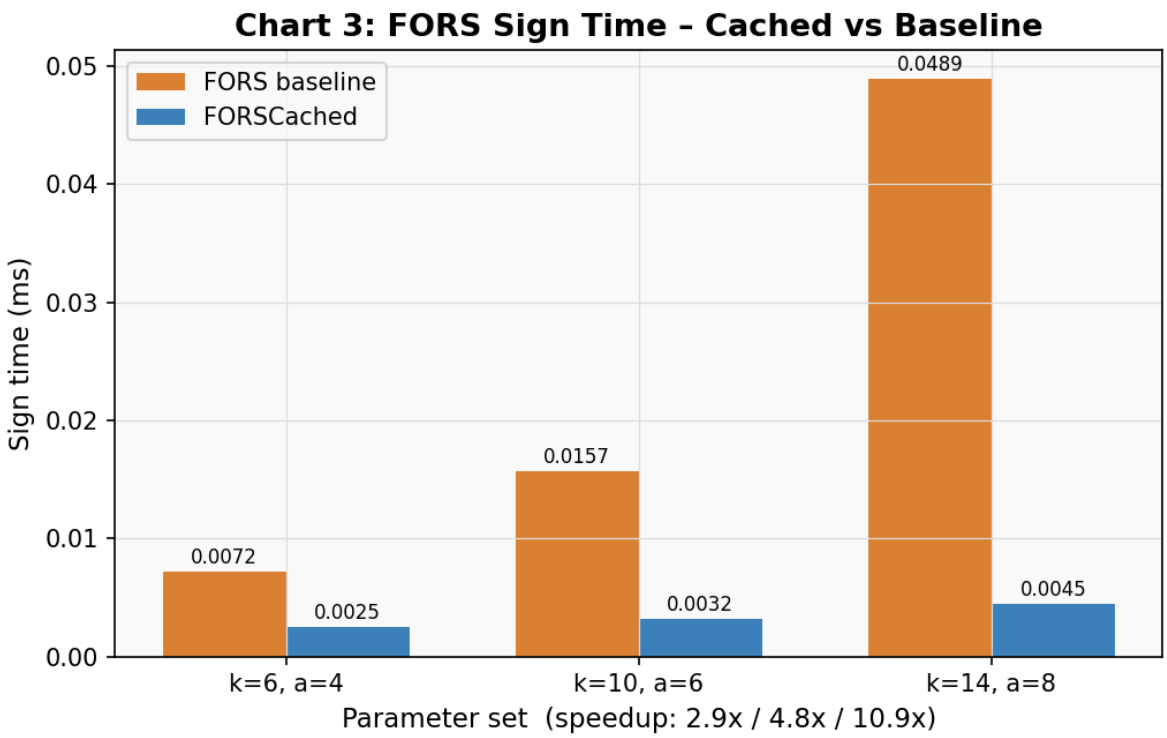

Figure 6. FORS signing time comparison between the baseline implementation and the cached implementation across multiple parameter sets.

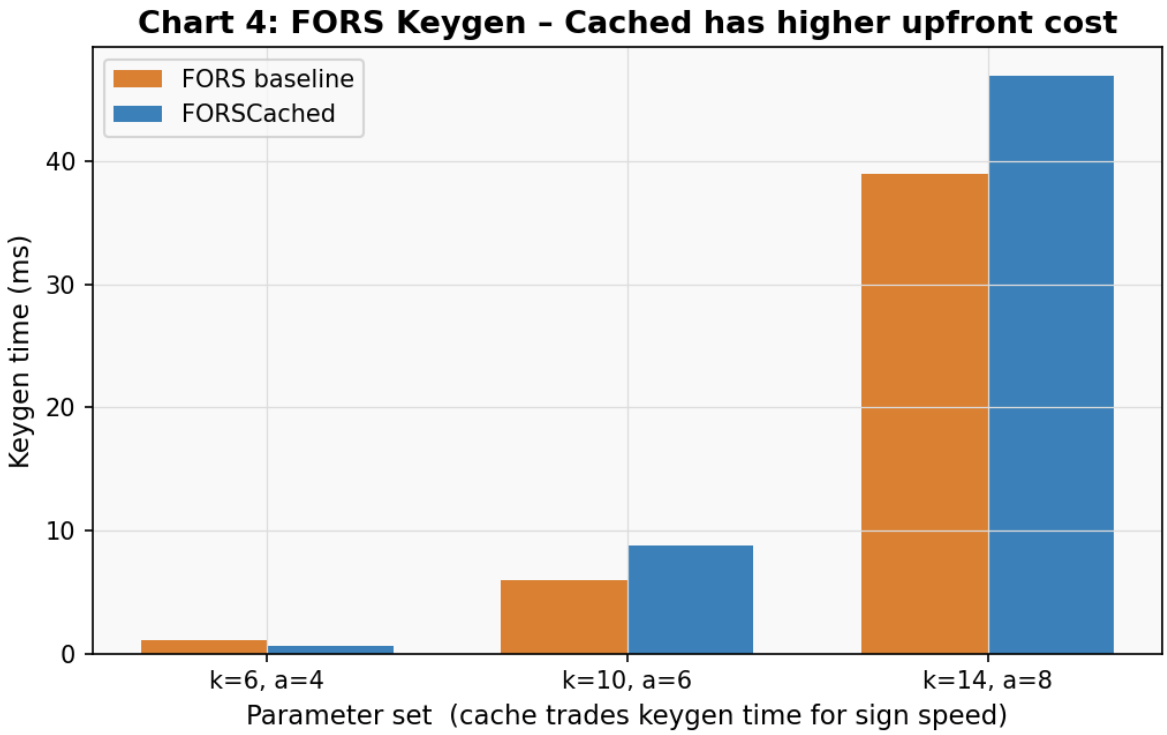

Figure 7. FORS key generation time comparison showing the higher upfront cost of the cached design.

Operation	Baseline avg	Baseline min	Baseline max	Cached avg	Cached min	Cached max
keygen	0.265 ms	0.229 ms	0.465 ms	0.607 ms	0.403 ms	0.973 ms
sign	0.026 ms	0.021 ms	0.066 ms	0.007 ms	0.004 ms	0.030 ms
verify	0.055 ms	0.038 ms	0.110 ms	0.041 ms	0.038 ms	0.073 ms

Table 1. k = 6, a = 4

Operation	Baseline avg	Baseline min	Baseline max	Cached avg	Cached min	Cached max
keygen	2.012 ms	1.444 ms	9.406 ms	4.277 ms	3.069 ms	7.777 ms
sign	0.034 ms	0.030 ms	0.081 ms	0.006 ms	0.005 ms	0.016 ms
verify	0.109 ms	0.083 ms	0.187 ms	0.092 ms	0.083 ms	0.169 ms

Table 2. $k = 10$, $a = 6$

Operation	Baseline avg	Baseline min	Baseline max	Cached avg	Cached min	Cached max
keygen	9.394 ms	7.977 ms	12.853 ms	27.638 ms	24.195 ms	35.890 ms
sign	0.093 ms	0.068 ms	0.397 ms	0.008 ms	0.006 ms	0.040 ms
verify	0.195 ms	0.146 ms	0.389 ms	0.151 ms	0.146 ms	0.209 ms

Table 3. $k = 14$, $a = 8$

5.5 System-Level Benchmark

A final benchmark was conducted for the complete SPHINCS+-style pipeline. The measured operations were key generation, signing, and verification under several parameter settings ranging from smaller to larger configurations.

At the system level, key generation was clearly the dominant cost. This is reasonable because larger parameter sets require more WOTS+ key pairs and more FORS structures to be created. Signing remained comparatively light because only one WOTS+ key pair and one FORS signing operation are used for each message. Verification also remained fairly stable because it reconstructs only the parts of the structure that are needed for the chosen signature.

These measurements suggest that the design is most suitable for settings in which setup happens relatively infrequently but signing and verification are performed many times afterwards.

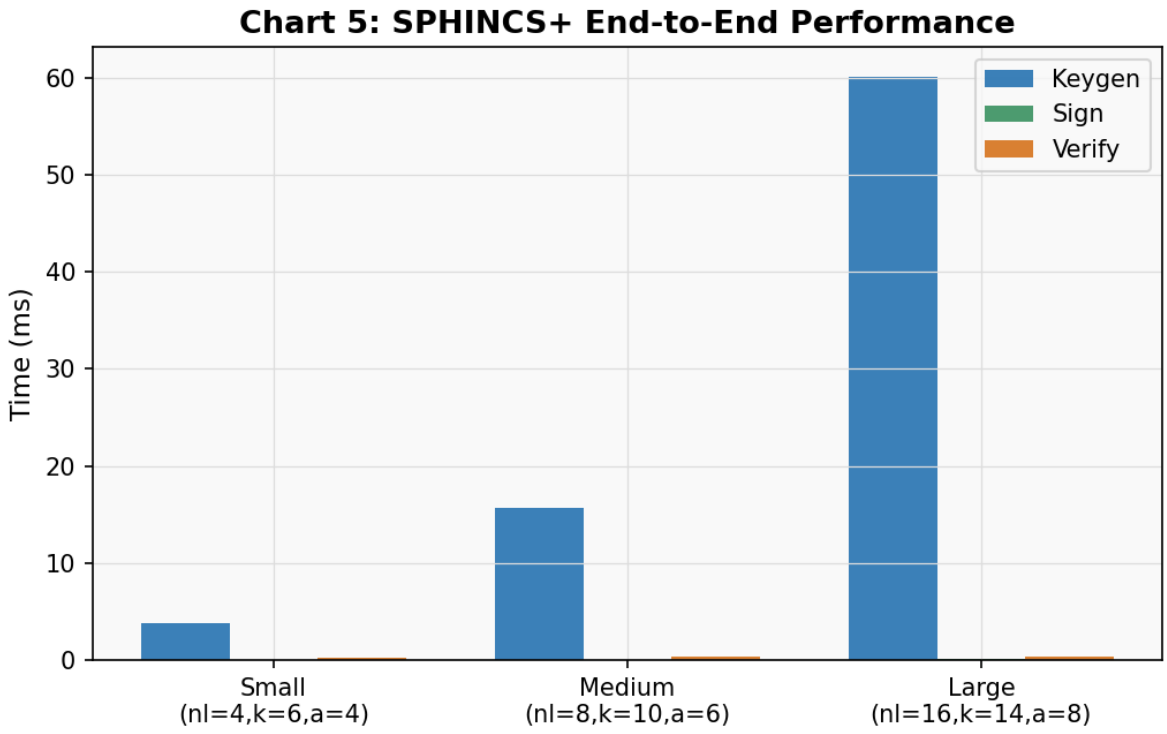

Figure 8. End-to-end SPHINCS+-style performance across small, medium, and large parameter settings.

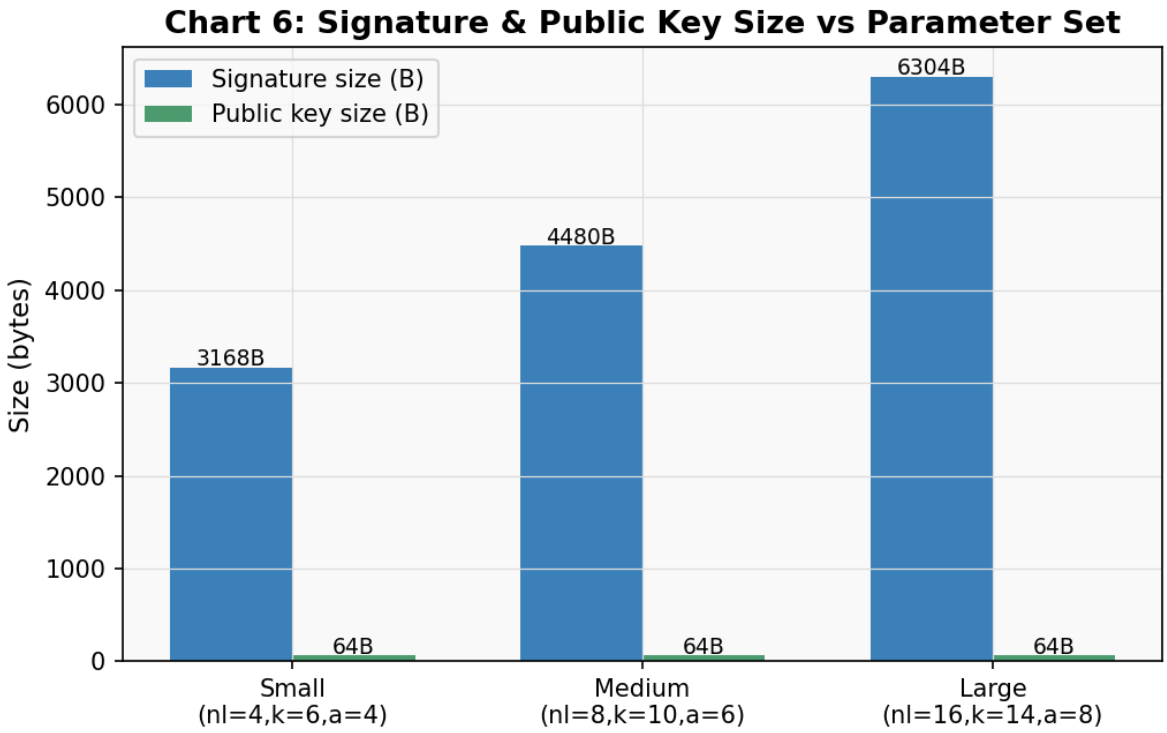

Figure 9. Signature size and public-key size across the three system-level parameter sets used in the project.

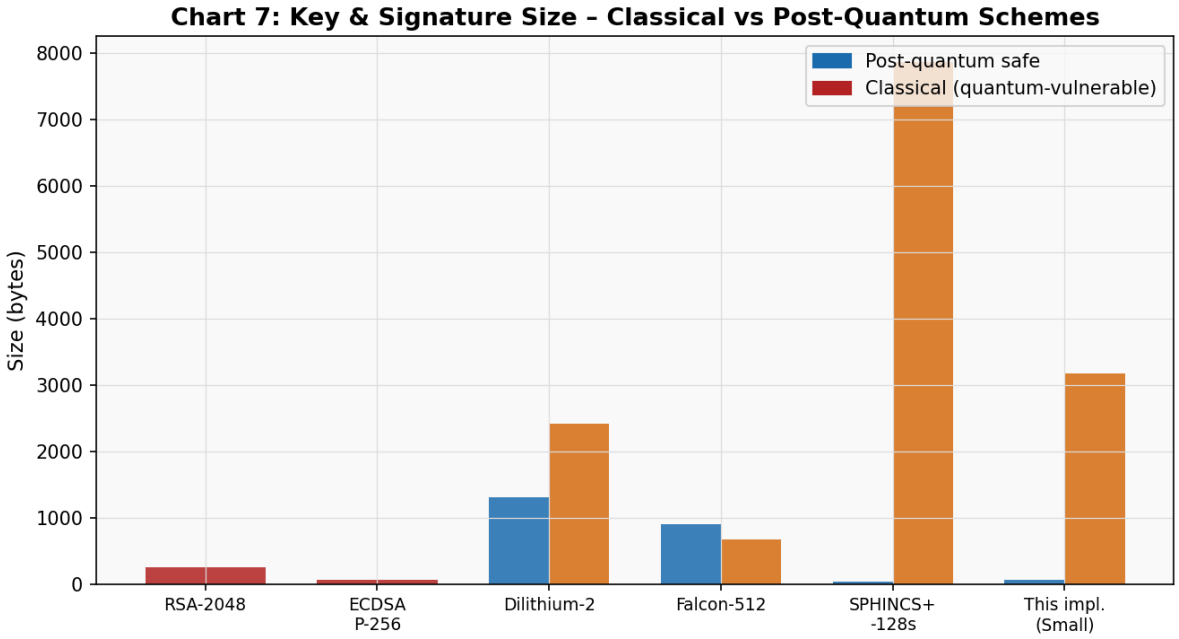

Figure 10. Key and signature size comparison between selected classical and post-quantum schemes, including this implementation.

6. Discussion

This project shows that a simplified SPHINCS+-style system can be built in a modular and experimentally meaningful way. Even though the implementation does not attempt to reproduce every detail of the production standard, it captures the main structural ideas that make SPHINCS+ interesting: a lower message-dependent signature layer, an upper one-time signature layer, and a Merkle-tree-based public authentication layer.

One of the clearest lessons from the project is that caching can provide real practical benefits when it is applied carefully. In WOTS+, caching intermediate chain values reduces repeated hash work during signing. In FORS, caching authentication paths significantly shortens the signing step, especially at larger parameter sizes. In both cases, the speed gain comes with a cost in setup time and memory usage, which is a classic trade-off in implementation design.

The experiments on the Winternitz parameter also highlight an important balance between efficiency and practicality. Larger values of w can improve raw performance and reduce signature length, but they also bring longer chains and higher cache cost. In this project, w = 16 turned out to be the most reasonable default because it balanced these factors well.

The negative result on parallelisation is also worth noting. In theory, independent hash chains look like a good match for parallel execution. In practice, however, the Python multiprocessing environment introduced too much overhead to make that worthwhile. This is a useful reminder that an optimisation that looks promising at the algorithmic level may still fail at the engineering level.

The system also has clear limitations. It is a simplified educational prototype rather than a standards-compliant implementation. It is written in Python, which is convenient for readability

but not ideal for performance. Some experiments were intentionally kept at modest parameter sizes so that the project would remain manageable. These limitations are acceptable in the context of a course project, but they also point to useful future work, such as larger-scale testing, stronger benchmarking infrastructure, and lower-level implementations in languages such as C, C++, or Rust.

7. Team Contributions

The project was completed as a collaborative group effort with a clear division of technical work. Overall coordination and top-level integration were handled at the system level through the design and implementation of the main SPHINCS+-style workflow. The WOTS+ module was developed and analysed separately, including caching, parameter analysis, and exploration of parallelisation. The Merkle tree module was implemented as the authentication layer for compressed WOTS+ public keys. The FORS module and the shared hash utilities were developed as the lower message-dependent signing layer. Benchmarking, testing, documentation, and report editing were also treated as dedicated responsibilities.

Beyond these primary roles, all members contributed to discussion, debugging, integration feedback, and review of the final system.

8. Conclusion

This project implemented and evaluated a simplified SPHINCS+-style post-quantum signature system based on four main components: shared hash utilities, WOTS+, Merkle-tree authentication, and FORS. These modules were integrated into a complete workflow for key generation, signing, and verification.

The system was validated through both module-level correctness tests and end-to-end pipeline testing. Benchmark experiments were also conducted for WOTS+, the Merkle tree, FORS, and the full system. The results showed that caching was an effective optimisation in both WOTS+

and FORS, while multiprocessing-based parallelisation was not beneficial in the present Python environment.

Taken together, the implementation and experiments show that the main design ideas behind hash-based post-quantum signatures can be expressed in a modular, testable, and reasonably efficient prototype. Although simplified, the project provides a useful proof of concept and a solid basis for future extension.

In addition to the benchmark and test scripts, the final project package also includes a runnable demo.py program, which provides a simple end-to-end demonstration of key generation, signing, and verification for the integrated SPHINCS+-style system.

Individual Contributions

Yixuan Wang (z5607523): Project coordination, integration of all modules, and implementation of the main SPHINCS+ workflow.

Shize Gao (z5603339): Design and implementation of the WOTS+ module.

Peiliang Zhao (z5539814): Design and implementation of the Merkle tree module.

Mingzi Chen (z5437121): Design and implementation of the simplified FORS and hash utility modules.

Kaiqi Shi (z5622283): Benchmarking, testing, documentation, and report preparation.